# Secure Persistent Group Chat

**Lab Assignment 4: Persistence, Encryption, Integrity, Authentication and Verification**

**Group of 3 Students**

**Group Head: Khethavath Sunil Naik 12341170**

**Member 2: Dosapati Ashiravdhan 12340710**

**Member 3: Dara Navya 12341170**

**Client URL for TA testing: http://172.17.0.6:8000**

**GitHub repository Group Head:** https://github.com/Sunil0012/Secure-Group-Chat.git

**Run the backend on the server machine and connect all laboratory clients to the same network.**

## 1. Problem and Objective

The earlier chat application delivered messages in real time but did not retain history or protect stored data. This update turns it into a secure persistent group chat. Users authenticate with an account, receive previous messages after login, and exchange live messages through the same WebSocket room.

## 2. Technology Stack

Python 3 standard library, SQLite, the cryptography package, HTML, CSS, JavaScript, TCP sockets, and the WebSocket protocol. The server remains dependency-light: only cryptography is installed from requirements.txt.

## 3. Requirement-to-Implementation Mapping

| | |
|---|---|
| **Messages stored in a database** | SQLite chat.db; messages table stores sender metadata, ciphertext, nonce, signature and timestamp. |
| **Previous chat history** | After successful authentication, the server decrypts and verifies each row and sends history events before live events. |
| **Messages not stored as plaintext** | AES-256-GCM encrypts the text before INSERT. The database stores ciphertext bytes only. |

| | |
|---|---|
| **Modification detected** | GCM authentication rejects altered ciphertext, nonce or authenticated metadata; a security event is shown and logged. |
| **Signing key pair per sender** | An Ed25519 private/public key pair is generated for each registered user. The private key is encrypted at rest. |
| **Signature and verification** | The sender signs canonical message metadata and text. The server verifies the signature after decrypting and before display/broadcast. |

### 4. Architecture and Message Flow

The backend listens on 0.0.0.0:8000 and serves both the client page and the /chat WebSocket endpoint. Each connection has a handler thread. Authenticated clients are maintained in a thread-safe set. SQLite operations are protected by a lock and each operation uses its own connection.

| | | |
|---|---|---|
| Browser / CLI clients | WebSocket /chat | Python server<br>Authentication<br>SQLite<br>AES-GCM<br>Ed25519 |

Sending: authenticate -> validate input -> encrypt with AES-GCM -> sign with Ed25519 -> store in SQLite -> broadcast verified event.

History: retrieve -> decrypt and authenticate -> verify sender public-key signature -> display only verified plaintext.

### 5. Authentication and Key Management

Registration creates a random salt, a PBKDF2-HMAC-SHA256 password hash, and an Ed25519 key pair. The public key is stored for verification. The private key is encrypted with AES-GCM using a 32-byte application master key. The master key is generated locally in chat_master.key or supplied through CHAT_MASTER_KEY and is excluded from version control.

## 6. Database Design

users(id, username, password_salt, password_hash, public_key, private_key_nonce, private_key_ciphertext, created_at)

messages(id, room_id, sender_id, sender, ciphertext, nonce, signature, timestamp)

security_events(id, message_id, reason, detected_at)

## 7. Setup and Client URL

Install Python 3 and run python -m pip install -r requirements.txt. Start the server with python server.py. Open http://SERVER_IP:8000 on each allotted machine. If the network address changes, use the active IPv4 address from ipconfig and allow TCP port 8000 through the firewall.

## 8. Demonstration and Verification

The implementation was tested with two WebSocket clients. The test registered a user, authenticated a second connection, sent a signed message, confirmed that the second connection received it from history, and flipped a byte in the stored ciphertext. On the next history read, AES-GCM rejected the row and the server generated a security alert instead of displaying plaintext. The test output was: Registration, authentication, persistence, history, broadcast, signatures, and tamper detection passed. For a manual demo, register two or more accounts, send messages, refresh or reconnect to show history, then stop the server and modify one ciphertext byte in chat.db. Restart and log in to show the red security alert.

## 9. Source Code

The attached .zip file contains all the code files

1. server.py
2. client.py
3. requirements.txt
4. index.html
5. README.md
6. Lab4_Report.pdf

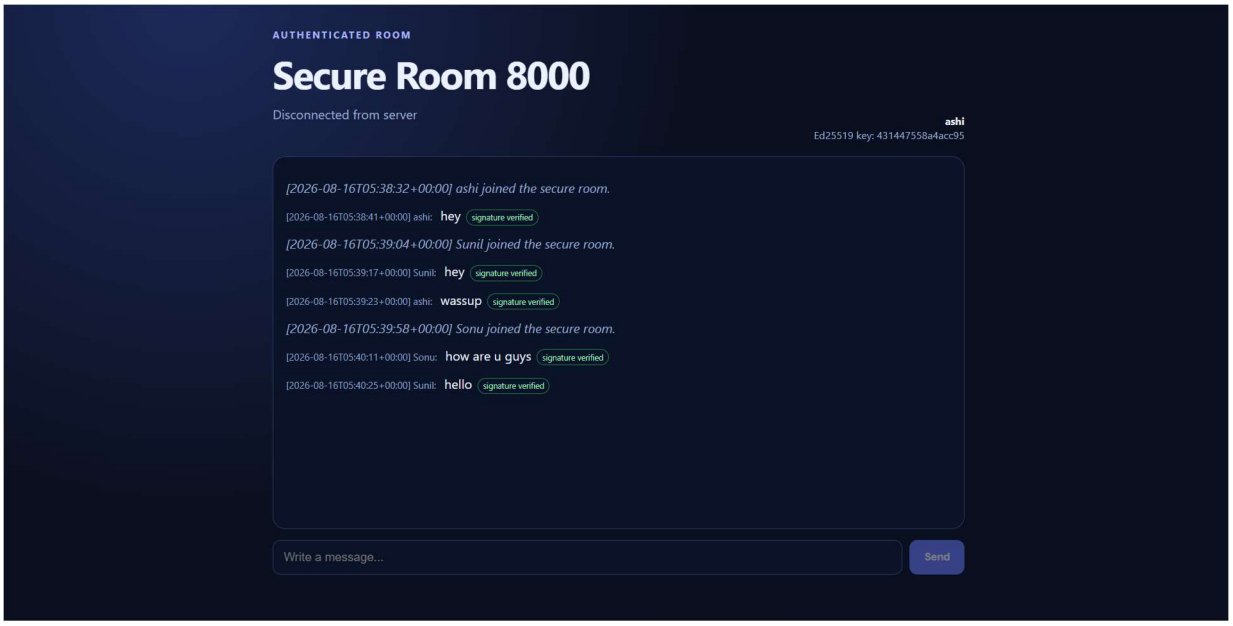


Fig 1: initial Chat without tampering the messages

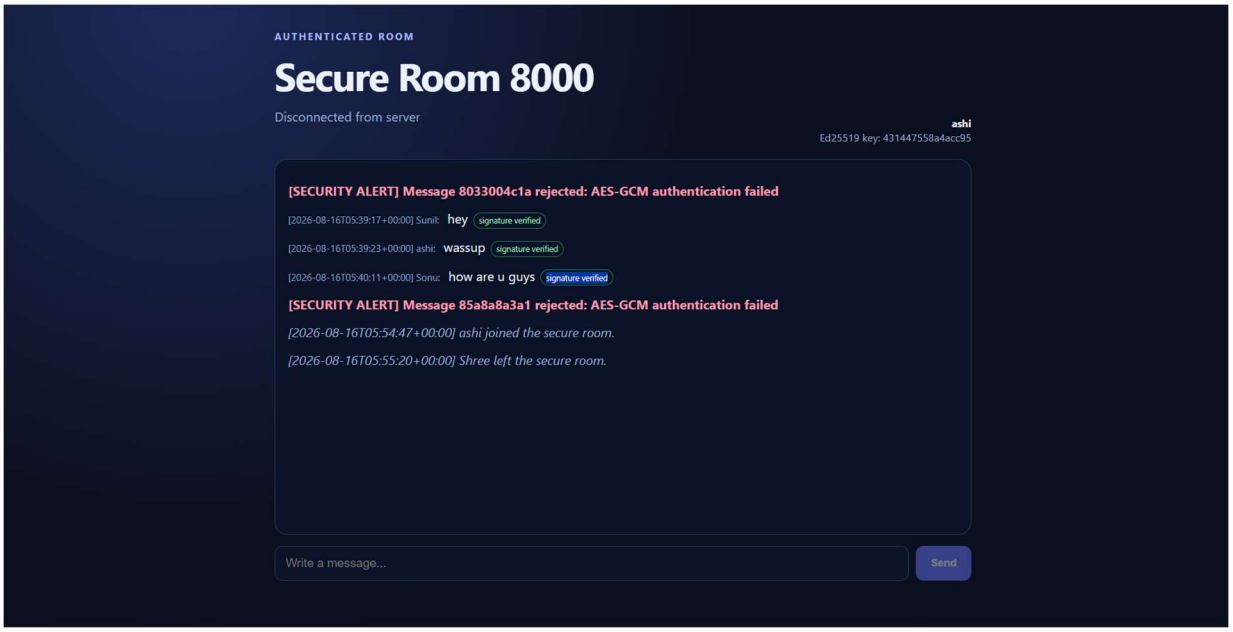


Fig 2: Chat with tampering the messages and detection

## 10. Contribution Report

| Student | Assigned contribution | Share |
|---|---|---|
| Group Head: Khethavath Sunil Naik | Architecture, integration, server deployment, SQLite persistence, Report | 33.34% |
| Member 2: Dosapati Ashirvadhan | AES-GCM encryption, integrity testing, client testing | 33.33% |
| Member 3: Dara Navya | Authentication, Ed25519 key management and verification | 33.33% |